

Task M2.T1P Parallel Matrix Multiplication

1. Overview

This report documents the design, parallelisation, and performance evaluation of a matrix multiplication program $C = A \times B$, implemented sequentially (matmul_seq.cpp) and in two parallel forms: std::thread (matmul_thread.cpp) and OpenMP (matmul_omp.cpp), as required by Task M2.T1P.

2. Program Design

2.1 Sequential implementation

matmul_seq.cpp uses the standard triple nested loop over i, j, k . A and B are $N \times N$ matrices of doubles, allocated on the heap and filled with random integers 0–99 using a fixed seed (42), so that A and B are identical across the sequential, std::thread, and OpenMP runs for a given N . This is what makes the checksum comparison in Section 4 a valid correctness check. Timing wraps only the multiply() call; initialisation and file output happen outside the timed region.

2.2 Compilation commands

```
g++ -O2 -std=c++17 matmul_seq.cpp -o matmul_seq.exe
g++ -O2 -std=c++17 -pthread matmul_thread.cpp -o matmul_thread.exe
g++ -O2 -std=c++17 -fopenmp matmul_omp.cpp -o matmul_omp.exe
```

All three compiled without errors or warnings

3. Decomposition and Parallelisation Plan

Each output element $C[i][j]$ depends only on row i of A and column j of B , and not on any other element of C , so different rows of C can be computed independently, with no data dependency between them and no need for locking, since each thread writes to a disjoint set of rows.

Both parallel programs use row-block (block-striped) decomposition: the N rows of C are split into contiguous blocks, one per thread. In matmul_thread.cpp this split is done explicitly with rowsPerThread = $N / \text{numThreads}$, and the first $(N \% \text{numThreads})$ threads absorbing one extra row each so the load stays balanced when N doesn't divide evenly. In matmul_omp.cpp, the same row-block split is achieved with #pragma omp parallel for schedule(static) over the outer i loop, letting OpenMP's runtime handle the partitioning, thread creation, and the implicit join, rather than doing it by hand.

3.1 Activities: parallel vs sequential

- Random initialisation of A and B : sequential (before timing starts, fixed seed 42)
- Matrix multiplication ($C = A \times B$): parallel, row-block decomposition across threads
- Thread creation (std::thread) / parallel region fork (OpenMP): part of the timed section, since spawning threads is a real cost of the parallel approach
- Thread join (std::thread) / implicit barrier (OpenMP): main thread waits for all workers before computing the checksum or writing output
- Checksum calculation and writing C to file: sequential, after timing stops

3.2 Why this decomposition

- No shared-state contention: each thread's output rows are disjoint, so no mutexes or atomics are needed
- Balanced load: remainder rows are distributed one-per-thread rather than all dumped on the last thread
- Coarse granularity (whole rows per thread, not individual cells) keeps thread-management overhead small relative to the work each thread does

4. Correctness Verification

Because A and B are generated with the same fixed seed in all three programs, the sequential, `std::thread`, and OpenMP runs should produce identical output matrices for the same N. Each program computes a checksum (sum of every element of C). The checksums below were read from each program's output for N = 800, 1600, and 2400, and match exactly in every case which including across every thread count tested (2, 4, and 8), confirming both parallel implementations compute the same result as the sequential baseline regardless of how many threads are used.

N	Sequential checksum	<code>std::thread</code> checksum	OpenMP checksum
800	1253730689227.0	1253730689227.0	1253730689227.0
1600	10023096198117.0	10023096198117.0	10023096198117.0
2400	33827346238363.0	33827346238363.0	33827346238363.0

5. Test Machine Specifications

Identified from `systeminfo` output:

Item	Value
Operating system	Windows 11 Home, Build 22631 (Git Bash / MINGW64)
System	Lenovo 82XW (LOQ series)
CPU model	Intel Core i7-13620H (13th Gen "Raptor Lake-H"), ~2.4 GHz base
Physical cores	10 (6 Performance-cores + 4 Efficient-cores)
Logical threads	16 (P-cores are hyperthreaded: 12 threads; E-cores are not: 4 threads)
RAM	16 GB (16,108 MB total physical memory)
Compiler & version	g++ (MinGW-w64), -O2 -std=c++17

6. Performance Evaluation

6.1 Methodology

Each program was run once per configuration at N = 800, 1600, and 2400, with the parallel versions run at 2, 4, and 8 threads. Execution time is the `time_ms` reported by each program (wrapping only the multiply step, including thread spawn/join for the parallel versions). Speedup is calculated as sequential time ÷ parallel time for the same N. This satisfies the brief's requirement of at least three matrix sizes and more than one thread count.

6.2 Results

N	Threads	Version	Time (ms)	Speedup vs seq
800	1	Sequential	341.630	Not applicable
800	2	<code>std::thread</code>	171.875	1.99x

N	Threads	Version	Time (ms)	Speedup vs seq
800	4	std::thread	87.666	3.90x
800	8	std::thread	62.087	5.50x
800	2	OpenMP	167.975	2.03x
800	4	OpenMP	98.071	3.48x
800	8	OpenMP	70.127	4.87x
1600	1	Sequential	5368.710	Not applicable
1600	2	std::thread	2409.470	2.23x
1600	4	std::thread	1068.912	5.02x
1600	8	std::thread	966.014	5.56x
1600	2	OpenMP	2506.166	2.14x
1600	4	OpenMP	1155.591	4.65x
1600	8	OpenMP	978.911	5.48x
2400	1	Sequential	61165.597	Not applicable
2400	2	std::thread	25510.320	2.40x
2400	4	std::thread	11956.320	5.12x
2400	8	std::thread	10024.889	6.10x
2400	2	OpenMP	25170.334	2.43x
2400	4	OpenMP	15199.862	4.02x
2400	8	OpenMP	10005.472	6.11x

6.3 Run evidence

```
giaki@BSFubuki: ~/NCa64 /c/Users/giaki/School/Sit_315/M2.TIP
$ ./matmul_seq.exe 800
Sequential Result:
N: 800
Threads: 1
Time: 341.630000 ms
Checksum: 1253730689227.000000000000000000
Output written to: seq_output.txt

giaki@BSFubuki: ~/NCa64 /c/Users/giaki/School/Sit_315/M2.TIP
$ ./matmul_seq.exe 1600
Sequential Result:
N: 1600
Threads: 1
Time: 5368.710200 ms
Checksum: 10023096198117.000000000000000000
Output written to: seq_output.txt

giaki@BSFubuki: ~/NCa64 /c/Users/giaki/School/Sit_315/M2.TIP
$ ./matmul_seq.exe 2400
Sequential Result:
N: 2400
Threads: 1
Time: 61165.596800 ms
Checksum: 33827346238363.000000000000000000
Output written to: seq_output.txt
```

Figure 1: Sequential runs, N = 800 / 1600 / 2400.

```
giaki@BSFubuki: ~/NCa64 /c/Users/giaki/School/Sit_315/M2.TIP
$ ./matmul_thread.exe 800 2
std::thread Result:
N: 800
Threads: 2
Rows per thread (approx): 400
Time: 171.874900 ms
Checksum: 1253730689227.000000000000000000
Output written to: thread_output.txt

giaki@BSFubuki: ~/NCa64 /c/Users/giaki/School/Sit_315/M2.TIP
$ ./matmul_thread.exe 1600 2
std::thread Result:
N: 1600
Threads: 2
Rows per thread (approx): 800
Time: 2409.469500 ms
Checksum: 10023096198117.000000000000000000
Output written to: thread_output.txt

giaki@BSFubuki: ~/NCa64 /c/Users/giaki/School/Sit_315/M2.TIP
$ ./matmul_thread.exe 2400 2
std::thread Result:
N: 2400
Threads: 2
Rows per thread (approx): 1200
Time: 25510.383200 ms
Checksum: 33827346238363.000000000000000000
Output written to: thread_output.txt
```

Figure 2: std::thread runs, 2 threads, N = 800 / 1600 / 2400.

```

giak@BSFubuki MINGW64 /c/Users/giak/School/Sit_315/M2.TIP
$ ./matmul_thread.exe 800 4
std::thread Result:
N: 800
Threads: 4
Rows per thread (approx): 200
Time: 87.666400 ms
Checksum: 1253730689227.000000000000000000
Output written to: thread_output.txt

giak@BSFubuki MINGW64 /c/Users/giak/School/Sit_315/M2.TIP
$ ./matmul_thread.exe 1600 4
std::thread Result:
N: 1600
Threads: 4
Rows per thread (approx): 400
Time: 1068.912200 ms
Checksum: 10023096198117.000000000000000000
Output written to: thread_output.txt

giak@BSFubuki MINGW64 /c/Users/giak/School/Sit_315/M2.TIP
$ ./matmul_thread.exe 2400 4
std::thread Result:
N: 2400
Threads: 4
Rows per thread (approx): 600
Time: 11956.320000 ms
Checksum: 33827346238363.000000000000000000
Output written to: thread_output.txt

```

Figure 3: std::thread runs, 4 threads, $N = 800 / 1600 / 2400$.

```

giak@BSFubuki MINGW64 /c/Users/giak/School/Sit_315/M2.TIP
$ ./matmul_thread.exe 800 8
std::thread Result:
N: 800
Threads: 8
Rows per thread (approx): 100
Time: 62.087500 ms
Checksum: 1253730689227.000000000000000000
Output written to: thread_output.txt

giak@BSFubuki MINGW64 /c/Users/giak/School/Sit_315/M2.TIP
$ ./matmul_thread.exe 1600 8
std::thread Result:
N: 1600
Threads: 8
Rows per thread (approx): 200
Time: 966.013700 ms
Checksum: 10023096198117.000000000000000000
Output written to: thread_output.txt

giak@BSFubuki MINGW64 /c/Users/giak/School/Sit_315/M2.TIP
$ ./matmul_thread.exe 2400 8
std::thread Result:
N: 2400
Threads: 8
Rows per thread (approx): 300
Time: 10024.888600 ms
Checksum: 33827346238363.000000000000000000
Output written to: thread_output.txt

```

Figure 4: std::thread runs, 8 threads, $N = 800 / 1600 / 2400$.

```

giak@BSFubuki MINGW64 /c/Users/giak/School/Sit_315/M2.TIP
$ ./matmul_omp.exe 800 2
OpenMP Result:
N: 800
Threads: 2
Time: 167.974700 ms
Checksum: 1253730689227.000000000000000000
Output written to: omp_output.txt

giak@BSFubuki MINGW64 /c/Users/giak/School/Sit_315/M2.TIP
$ ./matmul_omp.exe 1600 2
OpenMP Result:
N: 1600
Threads: 2
Time: 2106.165800 ms
Checksum: 10023096198117.000000000000000000
Output written to: omp_output.txt

giak@BSFubuki MINGW64 /c/Users/giak/School/Sit_315/M2.TIP
$ ./matmul_omp.exe 2400 2
OpenMP Result:
N: 2400
Threads: 2
Time: 23170.334400 ms
Checksum: 33827346238363.000000000000000000
Output written to: omp_output.txt

```

Figure 5: OpenMP runs, 2 threads, $N = 800 / 1600 / 2400$.

```

giak@BSFubuki MINGW64 /c/Users/giak/School/Sit_315/M2.TIP
$ ./matmul_omp.exe 800 4
OpenMP Result:
N: 800
Threads: 4
Time: 98.070900 ms
Checksum: 1253730689227.000000000000000000
Output written to: omp_output.txt

giak@BSFubuki MINGW64 /c/Users/giak/School/Sit_315/M2.TIP
$ ./matmul_omp.exe 1600 4
OpenMP Result:
N: 1600
Threads: 4
Time: 1155.591500 ms
Checksum: 10023096198117.000000000000000000
Output written to: omp_output.txt

giak@BSFubuki MINGW64 /c/Users/giak/School/Sit_315/M2.TIP
$ ./matmul_omp.exe 2400 4
OpenMP Result:
N: 2400
Threads: 4
Time: 15109.861800 ms
Checksum: 33827346238363.000000000000000000
Output written to: omp_output.txt

```

Figure 6: OpenMP runs, 4 threads, $N = 800 / 1600 / 2400$.

```

giaki@PSFubuki MINGW64 /c:/Users/giaki/School/Sit_315/M2.T1P
$ ./matmul_omp.exe 800 8
OpenMP Result:
N: 800
Threads: 8
Time: 70.127300 ms
Checksum: 1253730689227_000000000000000000
Output written to: omp_output.txt

giaki@PSFubuki MINGW64 /c:/Users/giaki/School/Sit_315/M2.T1P
$ ./matmul_omp.exe 1600 8
OpenMP Result:
N: 1600
Threads: 8
Time: 978.911100 ms
Checksum: 10023096198117_000000000000000000
Output written to: omp_output.txt

giaki@PSFubuki MINGW64 /c:/Users/giaki/School/Sit_315/M2.T1P
$ ./matmul_omp.exe 2400 8
OpenMP Result:
N: 2400
Threads: 8
Time: 10005.471000 ms
Checksum: 33827346238363_000000000000000000
Output written to: omp_output.txt

```

Figure 7: OpenMP runs, 8 threads, $N = 800 / 1600 / 2400$.

6.4 Parallel efficiency (speedup ÷ threads)

N	Threads	std::thread efficiency	OpenMP efficiency
800	2	99.4%	101.7%
800	4	97.4%	87.1%
800	8	68.8%	60.9%
1600	2	111.4%	107.1%
1600	4	125.6%	116.1%
1600	8	69.5%	68.6%
2400	2	119.9%	121.5%
2400	4	127.9%	100.6%
2400	8	76.3%	76.4%

Efficiency percentage formula:

speedup time = (sequential time / parallel time)

efficiency percentage = (speedup / thread) * 100%.

6.5 Discussion

Three clear trends emerge from the data:

- Efficiency peaks around 2 and 4 threads, often exceeding 100% (superlinear speedup). For example std::thread at $N=2400$ hits 127.9% efficiency at 4 threads. Speedup like this seem to be effect of cache behaviour: when the work is split across threads, each thread touches a smaller slice of A per call, which fits cache more favourably than the single-threaded version working through the entire matrix, so the parallel version isn't just splitting the same work four ways, it's also making better use of cache than the sequential baseline does.
- Efficiency drops sharply at 8 threads across every size and both libraries (down to roughly 61% to 76%, from over 100% at 4 threads). The machine is a hybrid-core CPU: 6 Performance-cores (hyperthreaded, 12 logical threads) plus 4 Efficient-cores (4 logical threads, no hyperthreading, and meaningfully slower per-thread than a P-core). Both matmul_thread.cpp and matmul_omp.cpp split rows into equal-sized blocks, which is a fair decomposition on a uniform CPU but not on this one: at 4 threads, the OS scheduler can likely place every worker on a P-core, so all four threads run at comparable speed. At 8 threads, some workers are very likely landing on the slower E-cores while others land on P-cores. Since every thread is given an equal share of rows, the whole computation has to wait at the join/barrier for whichever thread landed on the slowest core, which drags overall efficiency down even though the total logical thread count (16) hasn't been reached yet. This is a scheduling-cost effect specific to heterogeneous CPUs, distinct from plain thread-creation overhead.
- std::thread and OpenMP track each other closely at 2 and 8 threads, but diverge more at 4 threads which most noticeably at $N=2400$, 4 threads, where std::thread reaches 5.12x speedup (127.9% efficiency) but OpenMP only reaches 4.02x (100.6% efficiency). Since both implementations use the identical row-block decomposition, this gap is most likely down to OpenMP's runtime scheduling/fork overhead rather than a difference in the decomposition itself.

7. Stress Test

The assignment requires at least one stress test that either fails to compile, does not finish (DNF), or runs for an excessively long time, with machine specs and (if known) the reason.

matmul_omp.exe was run at $N = 16000$ with 8 threads. After waiting 10 minutes with no output, the run was manually interrupted (Ctrl+C) which result a DNF / excessive-wait case, on the machine specified in Section 5 (Intel Core i7-13620H, 16 GB RAM).

A terminal window with a black background and green text. The prompt is 'g:\ak1\OpenMP\ak1\21\Code4 /C/Users/giak1/school/511_313/02_T1P'. The command entered is '\$./matmul_omp.exe 16000 8'. The terminal is mostly empty, indicating no output.

Figure 8: matmul_omp.exe 16000 8, no output after 10 minutes, cancelled.

Why this is not finish in 10 minutes

- Compute scale: $N=16000$ is 6.67x the linear dimension of the largest size completed ($N=2400$), which is 296x more multiply-add operations under raw $O(N^3)$ scaling. The $N=2400$ OpenMP run at 8 threads took ~10.0 seconds, so scaling that by 296x alone predicts a minimum of at least 49 minutes or more.
- Memory footprint: three $N \times N$ double matrices (A, B, C) at $N=16000$ need $16000^2 \times 8$ bytes ≈ 2.05 GB each, ≈ 6.14 GB total. Against 16 GB of installed RAM (Section 5) shared with the OS and everything else already running, this is large enough to risk heavy paging to disk, which would slow the run further on top of the raw compute cost above.

8. Reflection: std::thread vs OpenMP

This task helped me understand how to divide a problem into smaller independent tasks for parallel programming. I learned that std::thread requires more manual thread management, while OpenMP makes parallelisation simpler. The results also showed me that using more threads does not always improve performance because of CPU architecture and scheduling.

9. Conclusion

Overall, both std::thread and OpenMP successfully improved the matrix multiplication performance while producing the correct results. The best performance was generally achieved with fewer threads, showing that effective parallel programming requires balancing workload, thread count, and system resources.